JMD ONLINE BOOK — GOD-LEVEL AUTO-BUILD PROMPT

Paste this entire file into Claude Code / Cursor / Codex / OpenCode CLI

Zero cost to build, deploy, and run. No credit card required.

MISSION

You are a senior full-stack engineer. Build a complete, production-grade online betting/gaming platform called **JMD Online Book** from scratch. Every decision must optimize for:

1. **Zero runtime cost** (free tiers only)
2. **Production quality** (auth, security, realtime, admin panel)
3. **Mobile-first** (PWA, touch-optimized, bottom nav)
4. **Southeast Asian UX patterns** (dark theme, gold/amber accent, bottom tabs, marquee ticker, quick-amount chips)

Do not skip any step. Do not use placeholder comments. Write all code fully.

FREE STACK (ZERO COST)

Layer	Tool	Free Tier
Framework	Next.js 15 (App Router)	Free
Database	Supabase PostgreSQL	500MB free

Auth	Supabase Auth (email OTP)	50,000 MAU free
Realtime	Supabase Realtime	200 concurrent free
File Storage	Supabase Storage	1GB free
Deployment	Vercel Hobby	Free forever
Email (OTP)	Resend	3,000 emails/month free
Styling	Tailwind CSS v4	Free
State	Zustand	Free
Validation	Zod	Free
CI/CD	GitHub + Vercel auto-deploy	Free

NEVER suggest or use: Twilio paid SMS, paid APIs, AWS, Firebase paid plans, Docker paid hosting, PlanetScale paid, or any service requiring a credit card past free tier.

STEP 0 — PROJECT BOOTSTRAP

Run this first:

```
npx create-next-app@latest jmd-online-book \
  --typescript --tailwind --eslint --app \
  --src-dir --import-alias "@/*" --turbopack

cd jmd-online-book

npm install \
  @supabase/supabase-js \
  @supabase/ssr \
  zustand \
  zod \
  react-hook-form \
  @hookform/resolvers \
  framer-motion \
  react-hot-toast \
  react-icons \
  recharts \
  date-fns \
  clsx \
```

```
  tailwind-merge \
  resend \
  lucide-react

npm install -D @types/node
```

STEP 1 — SUPABASE PROJECT SETUP

1a. Create Supabase project

Go to supabase.com → New project → Note: URL, anon key, service role key.

1b. Environment variables

Create `.env.local`:

```
NEXT_PUBLIC_SUPABASE_URL=your_supabase_url
NEXT_PUBLIC_SUPABASE_ANON_KEY=your_anon_key
SUPABASE_SERVICE_ROLE_KEY=your_service_role_key
RESEND_API_KEY=your_resend_key
NEXT_PUBLIC_APP_NAME="JMD Online Book"
NEXT_PUBLIC_APP_URL=http://localhost:3000
OTP_SECRET=generate_a_random_32char_string_here
```

Add same vars to Vercel dashboard later.

1c. Full database schema

Run this SQL in Supabase SQL editor:

```
-- Enable UUID extension
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";

-- USERS (extends Supabase auth.users)
CREATE TABLE public.profiles (
  id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
  phone VARCHAR(20) UNIQUE,
  full_name VARCHAR(100),
  role VARCHAR(20) DEFAULT 'user' CHECK (role IN ('user', 'agent', 'admin')),
  balance DECIMAL(12,2) DEFAULT 0.00,
  total_deposited DECIMAL(12,2) DEFAULT 0.00,
  total_withdrawn DECIMAL(12,2) DEFAULT 0.00,
  referral_code VARCHAR(20) UNIQUE DEFAULT UPPER(SUBSTR(MD5(RANDOM()::TEXT), 1, 8
```

```
  referred_by UUID REFERENCES public.profiles(id),
  is_active BOOLEAN DEFAULT TRUE,
  is_verified BOOLEAN DEFAULT FALSE,
  avatar_url TEXT,
  last_login_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- TRANSACTIONS
CREATE TABLE public.transactions (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id UUID NOT NULL REFERENCES public.profiles(id),
  type VARCHAR(20) NOT NULL CHECK (type IN ('deposit','withdraw','bet','win','bon
  amount DECIMAL(12,2) NOT NULL,
  balance_before DECIMAL(12,2),
  balance_after DECIMAL(12,2),
  status VARCHAR(20) DEFAULT 'pending' CHECK (status IN ('pending','approved','re
  payment_method VARCHAR(50),
  payment_reference VARCHAR(100),
  screenshot_url TEXT,
  upi_id VARCHAR(100),
  bank_account VARCHAR(50),
  ifsc_code VARCHAR(20),
  account_holder VARCHAR(100),
  admin_note TEXT,
  approved_by UUID REFERENCES public.profiles(id),
  approved_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- GAMES
CREATE TABLE public.games (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  name VARCHAR(100) NOT NULL,
  provider VARCHAR(100) NOT NULL,
  category VARCHAR(50) NOT NULL CHECK (category IN ('sports','casino','lottery','
  thumbnail_url TEXT,
  launch_url TEXT,
  description TEXT,
  is_active BOOLEAN DEFAULT TRUE,
  is_featured BOOLEAN DEFAULT FALSE,
  sort_order INT DEFAULT 0,
  min_bet DECIMAL(10,2) DEFAULT 10,
  max_bet DECIMAL(10,2) DEFAULT 100000,
  tags TEXT[],
```

```
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- BETS
CREATE TABLE public.bets (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id UUID NOT NULL REFERENCES public.profiles(id),
  game_id UUID NOT NULL REFERENCES public.games(id),
  amount DECIMAL(12,2) NOT NULL,
  odds DECIMAL(8,4),
  potential_win DECIMAL(12,2),
  result VARCHAR(20) CHECK (result IN ('pending','win','lose','draw','void')),
  payout DECIMAL(12,2),
  settled_at TIMESTAMPTZ,
  metadata JSONB DEFAULT '{}',
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- REFERRAL COMMISSIONS
CREATE TABLE public.commissions (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  agent_id UUID NOT NULL REFERENCES public.profiles(id),
  player_id UUID NOT NULL REFERENCES public.profiles(id),
  transaction_id UUID REFERENCES public.transactions(id),
  amount DECIMAL(12,2) NOT NULL,
  rate DECIMAL(5,4) DEFAULT 0.05,
  type VARCHAR(20) DEFAULT 'deposit',
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- NOTIFICATIONS
CREATE TABLE public.notifications (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id UUID NOT NULL REFERENCES public.profiles(id),
  title VARCHAR(200) NOT NULL,
  body TEXT NOT NULL,
  type VARCHAR(50) DEFAULT 'info',
  is_read BOOLEAN DEFAULT FALSE,
  metadata JSONB DEFAULT '{}',
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- SITE SETTINGS
CREATE TABLE public.site_settings (
  key VARCHAR(100) PRIMARY KEY,
  value TEXT,
  description TEXT,
```

```
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- OTP TABLE (for phone OTP via email fallback)
CREATE TABLE public.otp_tokens (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  identifier VARCHAR(100) NOT NULL,
  token VARCHAR(6) NOT NULL,
  expires_at TIMESTAMPTZ NOT NULL,
  is_used BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- PAYMENT METHODS CONFIG
CREATE TABLE public.payment_methods (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  name VARCHAR(100) NOT NULL,
  type VARCHAR(50) NOT NULL,
  details JSONB DEFAULT '{}',
  is_active BOOLEAN DEFAULT TRUE,
  sort_order INT DEFAULT 0,
  min_amount DECIMAL(10,2) DEFAULT 100,
  max_amount DECIMAL(10,2) DEFAULT 100000,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- INDEXES
CREATE INDEX idx_transactions_user_id ON public.transactions(user_id);
CREATE INDEX idx_transactions_status ON public.transactions(status);
CREATE INDEX idx_transactions_created_at ON public.transactions(created_at DESC);
CREATE INDEX idx_bets_user_id ON public.bets(user_id);
CREATE INDEX idx_notifications_user_id ON public.notifications(user_id, is_read);
CREATE INDEX idx_otp_identifier ON public.otp_tokens(identifier, expires_at);

-- ROW LEVEL SECURITY
ALTER TABLE public.profiles ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.transactions ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.bets ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.notifications ENABLE ROW LEVEL SECURITY;

-- POLICIES: profiles
CREATE POLICY "Users can view own profile" ON public.profiles FOR SELECT USING (a
CREATE POLICY "Users can update own profile" ON public.profiles FOR UPDATE USING
CREATE POLICY "Admins can view all profiles" ON public.profiles FOR ALL USING (
  EXISTS (SELECT 1 FROM public.profiles WHERE id = auth.uid() AND role = 'admin')
);
CREATE POLICY "Public profiles insert" ON public.profiles FOR INSERT WITH CHECK (
```

```
-- POLICIES: transactions
CREATE POLICY "Users view own transactions" ON public.transactions FOR SELECT USI
CREATE POLICY "Users create own transactions" ON public.transactions FOR INSERT W
CREATE POLICY "Admins manage all transactions" ON public.transactions FOR ALL USI
  EXISTS (SELECT 1 FROM public.profiles WHERE id = auth.uid() AND role IN ('admin
);

-- POLICIES: notifications
CREATE POLICY "Users view own notifications" ON public.notifications FOR ALL USIN

-- POLICIES: bets
CREATE POLICY "Users view own bets" ON public.bets FOR SELECT USING (auth.uid() =
CREATE POLICY "Users create bets" ON public.bets FOR INSERT WITH CHECK (auth.uid(

-- Games are public read
CREATE POLICY "Anyone can view active games" ON public.games FOR SELECT USING (is

-- REALTIME (enable for live balance/notification updates)
ALTER PUBLICATION supabase_realtime ADD TABLE public.notifications;
ALTER PUBLICATION supabase_realtime ADD TABLE public.transactions;
ALTER PUBLICATION supabase_realtime ADD TABLE public.profiles;

-- FUNCTIONS
CREATE OR REPLACE FUNCTION public.handle_new_user()
RETURNS TRIGGER AS $$
BEGIN
  INSERT INTO public.profiles (id, full_name)
  VALUES (NEW.id, NEW.raw_user_meta_data->>'full_name');
  RETURN NEW;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

CREATE TRIGGER on_auth_user_created
AFTER INSERT ON auth.users
FOR EACH ROW EXECUTE FUNCTION public.handle_new_user();

-- Balance update function (atomic, safe)
CREATE OR REPLACE FUNCTION public.update_balance(
  p_user_id UUID,
  p_amount DECIMAL,
  p_type TEXT
)
RETURNS JSONB AS $$
DECLARE
  v_current_balance DECIMAL;
  v_new_balance DECIMAL;
```

```
BEGIN
  SELECT balance INTO v_current_balance FROM public.profiles WHERE id = p_user_id

  IF p_type IN ('withdraw','bet') AND v_current_balance < ABS(p_amount) THEN
    RETURN jsonb_build_object('success', false, 'error', 'Insufficient balance');
  END IF;

  v_new_balance := v_current_balance + p_amount;

  UPDATE public.profiles SET balance = v_new_balance, updated_at = NOW() WHERE id

  RETURN jsonb_build_object('success', true, 'balance', v_new_balance);
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

-- DEFAULT DATA
INSERT INTO public.site_settings (key, value, description) VALUES
  ('site_name', 'JMD Online Book', 'Platform name'),
  ('min_deposit', '100', 'Minimum deposit amount in INR'),
  ('min_withdraw', '200', 'Minimum withdrawal amount'),
  ('max_withdraw_daily', '50000', 'Daily withdrawal limit'),
  ('referral_commission_rate', '0.05', 'Agent commission rate (5%)'),
  ('maintenance_mode', 'false', 'Enable maintenance mode'),
  ('welcome_bonus', '0', 'Welcome bonus amount'),
  ('upi_id', 'jmdonlinebook@upi', 'Company UPI ID for deposits'),
  ('whatsapp_support', '+919999999999', 'WhatsApp support number'),
  ('announcement', 'Welcome to JMD Online Book! Fast withdrawals guaranteed.', 'M

INSERT INTO public.payment_methods (name, type, details, sort_order) VALUES
  ('UPI', 'upi', '{"upi_id": "jmdonlinebook@upi", "qr_url": ""}', 1),
  ('PhonePe', 'upi', '{"upi_id": "jmdonlinebook@ybl"}', 2),
  ('Google Pay', 'upi', '{"upi_id": "jmdonlinebook@okaxis"}', 3),
  ('Bank Transfer', 'bank', '{"bank_name":"HDFC Bank","account":"1234567890","ifs

INSERT INTO public.games (name, provider, category, is_featured, sort_order, tags
  ('IPL Cricket', 'JMD Sports', 'sports', true, 1, ARRAY['cricket','live','popula
  ('Teen Patti', 'JMD Casino', 'cards', true, 2, ARRAY['cards','casino','popular'
  ('Andar Bahar', 'JMD Casino', 'cards', true, 3, ARRAY['cards','casino']),
  ('Roulette', 'JMD Casino', 'casino', false, 4, ARRAY['casino','roulette']),
  ('Dragon Tiger', 'JMD Casino', 'casino', false, 5, ARRAY['cards','casino']),
  ('Lottery 3D', 'JMD Luck', 'lottery', false, 6, ARRAY['lottery']),
  ('Football Bet', 'JMD Sports', 'sports', false, 7, ARRAY['football','sports']),
  ('Horse Racing', 'JMD Sports', 'sports', false, 8, ARRAY['horse','sports']),
  ('Color Prediction', 'JMD Games', 'other', true, 9, ARRAY['color','popular']),
  ('Aviator', 'JMD Games', 'casino', true, 10, ARRAY['aviator','trending']);
```

STEP 2 — SUPABASE CLIENT SETUP

src/lib/supabase/client.ts

```
import { createBrowserClient } from '@supabase/ssr'
import type { Database } from '@/types/database'

export function createClient() {
  return createBrowserClient<Database>(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!
  )
}
```

src/lib/supabase/server.ts

```
import { createServerClient } from '@supabase/ssr'
import { cookies } from 'next/headers'
import type { Database } from '@/types/database'

export async function createClient() {
  const cookieStore = await cookies()
  return createServerClient<Database>(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll: () => cookieStore.getAll(),
        setAll: (cookiesToSet) => {
          cookiesToSet.forEach(({ name, value, options }) =>
            cookieStore.set(name, value, options)
          )
        },
      },
    }
  )
}
```

src/lib/supabase/admin.ts

```
import { createClient } from '@supabase/supabase-js'
import type { Database } from '@/types/database'
```

```
export function createAdminClient() {
  return createClient<Database>(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.SUPABASE_SERVICE_ROLE_KEY!,
    { auth: { autoRefreshToken: false, persistSession: false } }
  )
}
```

STEP 3 — TYPES

src/types/database.ts

Generate full TypeScript types matching the schema above. Include:

- `Database` interface with `public.Tables` for all tables
- `Profile`, `Transaction`, `Game`, `Bet`, `Notification`, `Commission` types
- Enum types for roles, statuses, transaction types
- `ApiResponse<T>` generic wrapper

STEP 4 — MIDDLEWARE (Auth Guard)

src/middleware.ts

```
import { createServerClient } from '@supabase/ssr'
import { NextResponse, type NextRequest } from 'next/server'

const PUBLIC_ROUTES = ['/login', '/register', '/forgot-password', '/verify-otp']
const ADMIN_ROUTES = ['/admin']
const AGENT_ROUTES = ['/agent']

export async function middleware(request: NextRequest) {
  let supabaseResponse = NextResponse.next({ request })

  const supabase = createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll: () => request.cookies.getAll(),
```

```
        setAll: (cookiesToSet) => {
          cookiesToSet.forEach(({ name, value, options }) =>
            supabaseResponse.cookies.set(name, value, options)
          )
        },
      },
    }
  )

  const { data: { user } } = await supabase.auth.getUser()
  const pathname = request.nextUrl.pathname

  if (!user && !PUBLIC_ROUTES.some(r => pathname.startsWith(r))) {
    return NextResponse.redirect(new URL('/login', request.url))
  }

  if (user && PUBLIC_ROUTES.some(r => pathname.startsWith(r))) {
    return NextResponse.redirect(new URL('/home', request.url))
  }

  if (user && ADMIN_ROUTES.some(r => pathname.startsWith(r))) {
    const { data: profile } = await supabase
      .from('profiles')
      .select('role')
      .eq('id', user.id)
      .single()
    if (!profile || profile.role !== 'admin') {
      return NextResponse.redirect(new URL('/home', request.url))
    }
  }

  return supabaseResponse
}

export const config = {
  matcher: ['/((?!_next/static|_next/image|favicon.ico|.*\\.(?:svg|png|jpg|jpeg|g
}
```

STEP 5 — ZUSTAND STORES

`src/store/useAuthStore.ts`

Store: `user` , `profile` , `isLoading` , `setUser` , `setProfile` , `signOut` , `updateBalance`

`src/store/useNotificationStore.ts`

Store: `notifications`, `unreadCount`, `markRead`, `addNotification`

`src/store/useAppStore.ts`

Store: `siteSettings`, `paymentMethods`, `announcement`, `maintenanceMode`

Each store uses `persist` middleware from zustand for offline resilience.

STEP 6 — API ROUTES

Build these server-side API routes. All use Zod validation. All return `{ data, error }` shape.

`/api/auth/send-otp` [POST]

- Accept: `{ email: string }` OR `{ phone: string }`
- Generate 6-digit OTP, store in `otp_tokens` table with 10min expiry
- Send via Resend email (free) to user's email
- If phone: format as `+91{phone}@jmdonline.internal` virtual email for OTP lookup
- Rate limit: max 3 OTPs per identifier per 15 minutes (check DB)
- Return: `{ success: true, message: "OTP sent" }`

`/api/auth/verify-otp` [POST]

- Accept: `{ identifier: string, token: string }`
- Validate OTP from DB, check not expired, not used
- Mark as used
- Return: `{ success: true }` or error

`/api/wallet/deposit` [POST] (authenticated)

- Accept: `{ amount, payment_method, upi_id, screenshot_url, reference }`
- Validate with Zod: amount >= site_settings.min_deposit
- Create transaction with status `pending`

- Send notification to admin
- Return transaction ID

`/api/wallet/withdraw` [POST] (authenticated)

- Accept: `{ amount, bank_account, ifsc_code, account_holder, upi_id }`
- Check user balance >= amount + min_withdraw
- Create transaction with status `pending`
- Return transaction ID

`/api/wallet/upload-screenshot` [POST] (authenticated)

- Accept: FormData with image file
- Upload to Supabase Storage bucket `screenshots` (public bucket)
- Return: `{ url: string }`

`/api/admin/transactions/approve` [POST] (admin only)

- Accept: `{ transaction_id, action: 'approve'|'reject', note? }`
- Use `update_balance` DB function atomically
- Update transaction status
- Create notification for user
- If deposit: update `total_deposited`
- If referral exists: create commission record and credit agent

`/api/admin/users` [GET] (admin only)

- Query params: `page, limit, search, role, status`
- Return paginated user list with balance, transaction counts

`/api/admin/dashboard/stats` [GET] (admin only)

- Return: total users, today's deposits, today's withdrawals, pending count, total balance in system, 7-day chart data

`/api/referral/tree` [GET] (authenticated)

- Return user's referral tree (direct referrals + their referrals, 2 levels)
- Include total commission earned

STEP 7 — FULL PAGE IMPLEMENTATIONS

App Layout Structure:

```
src/app/
├── (auth)/
│   ├── login/page.tsx          ← Phone/email + password login
│   ├── register/page.tsx       ← Full registration with referral
│   ├── verify-otp/page.tsx     ← 6-digit OTP input with countdown
│   └── forgot-password/page.tsx
├── (main)/
│   ├── layout.tsx              ← Header + MobileNav
│   ├── home/page.tsx           ← Balance card + games + marquee
│   ├── wallet/page.tsx         ← Balance + recent transactions
│   ├── deposit/page.tsx        ← Amount chips + method + screenshot upload
│   ├── withdraw/page.tsx       ← Amount + bank details form
│   ├── transactions/page.tsx   ← Paginated history with filters
│   ├── games/
│   │   ├── page.tsx            ← Game grid with category filter
│   │   └── [id]/page.tsx       ← Game detail / launch
│   ├── profile/page.tsx        ← User info + stats + settings
│   ├── referral/page.tsx       ← Referral code + tree + commission
│   └── notifications/page.tsx
└── (admin)/
    ├── layout.tsx              ← Admin sidebar layout
    ├── dashboard/page.tsx      ← Stats cards + charts + recent activity
    ├── users/
    │   ├── page.tsx            ← User table with search/filter
    │   └── [id]/page.tsx       ← User detail + manual balance adjustment
    ├── transactions/
    │   ├── page.tsx            ← Pending deposits/withdrawals queue
    │   └── [id]/page.tsx       ← Transaction detail + approve/reject
    ├── games/page.tsx          ← CRUD for games
    ├── payments/page.tsx       ← Payment method config
    └── settings/page.tsx       ← Site settings editor
```

PAGE SPECS — implement each fully:

`(auth)/login/page.tsx`

- Phone number input (auto-format with +91 prefix)
- Password field with show/hide toggle
- “Login with OTP” toggle (sends OTP to email linked to phone)
- Remember me checkbox
- Redirect to `/home` on success
- Error handling with toast

(auth)/register/page.tsx

- Full name, phone, email (optional), password, confirm password
- Referral code field (pre-fill from URL param `?ref=CODE`)
- Terms checkbox
- On submit: create Supabase auth user → trigger inserts profile
- Auto-login after registration

(auth)/verify-otp/page.tsx

- 6 individual digit input boxes (auto-advance on type, backspace to go back)
- 60-second countdown with “Resend” button
- Auto-submit when all 6 filled
- Show destination (masked phone/email)

(main)/home/page.tsx

- Hero balance card (gradient gold → amber) with deposit/withdraw buttons
- Animated marquee announcement from site_settings
- Quick action grid: Deposit, Withdraw, Results, Support
- Featured games horizontal scroll
- “Hot” badge on trending games
- Bottom safe area for mobile nav

(main)/deposit/page.tsx

- Amount input with preset chips: ₹100, ₹200, ₹500, ₹1K, ₹2K, ₹5K, ₹10K
- Payment method selector (cards with icon + name)
- After method selected: show company UPI ID / bank details to pay
- Screenshot upload button → calls `/api/wallet/upload-screenshot`
- UTR/reference number input
- Submit creates pending transaction
- Show "Pending approval" screen after submit

`(main)/withdraw/page.tsx`

- Current balance display
- Amount input with max button
- Bank details form: Account Number, IFSC, Account Holder Name OR UPI ID (toggle)
- Save details checkbox (persist to profile)
- Minimum withdraw warning from settings
- Submit creates pending withdrawal

`(main)/transactions/page.tsx`

- Tab filter: All / Deposits / Withdrawals / Bets
- Status filter pills: All / Pending / Completed / Rejected
- Infinite scroll (load 20 at a time)
- Each row: type icon, amount (colored), status badge, date, reference

`(main)/referral/page.tsx`

- Your referral code in big display with copy button
- Share link generator: `{APP_URL}/register?ref={code}`
- Stats: Total referred, Active players, Total commission earned
- Commission history table
- "How it works" section: referral → deposit → 5% commission credited

(admin)/dashboard/page.tsx

- Stats cards: Total Users, Today Deposits (₹), Today Withdrawals (₹), Pending Approvals
- Line chart (Recharts): 7-day deposit vs withdrawal trend
- Pie chart: User roles distribution
- Recent transactions table (last 10, with approve button)
- Pending approvals counter with alert if > 0

(admin)/transactions/page.tsx

- Two tabs: Pending Deposits / Pending Withdrawals
- Each card shows: User name+phone, Amount, Method, Reference, Screenshot (clickable)
- Approve button (green) + Reject button (red) with confirmation modal
- Rejection requires note
- Real-time updates via Supabase subscription

STEP 8 — REUSABLE COMPONENTS

Build these components fully (not stubs):

src/components/ui/

- `Button.tsx` — variants: gold, accent, ghost, danger; sizes: sm, md, lg; loading state with spinner
- `Input.tsx` — label, error, icon left/right, helper text
- `Modal.tsx` — backdrop blur, slide-up animation (framer-motion), close on backdrop click
- `Badge.tsx` — status badges: pending (yellow), approved (green), rejected (red), completed (blue)
- `Loader.tsx` — full screen and inline variants with gold spinner
- `BottomSheet.tsx` — slide-up sheet for mobile (framer-motion drag dismiss)

- `Avatar.tsx` — with initials fallback and online indicator
- `AmountChips.tsx` — preset amount selection chips
- `OTPInput.tsx` — 6-box OTP input with auto-advance

src/components/layout/

- `Header.tsx` — logo, balance display (from store), notification bell with unread count, support icon
- `MobileNav.tsx` — 5 tab icons: Home, Wallet, Games, History, Profile. Active state with amber color and scale
- `AdminSidebar.tsx` — collapsible sidebar for admin panel

src/components/wallet/

- `BalanceCard.tsx` — animated balance with refresh button
- `TransactionItem.tsx` — single transaction row with icon, color-coded amount

src/components/games/

- `GameCard.tsx` — thumbnail, name, provider, HOT/NEW badges, tap animation
- `GameGrid.tsx` — responsive 3-column grid with category filter tabs

STEP 9 — REALTIME SUBSCRIPTIONS

In the main layout, set up Supabase Realtime subscriptions:

```
// Listen for balance changes
supabase
  .channel('profile-changes')
  .on('postgres_changes', {
    event: 'UPDATE',
    schema: 'public',
    table: 'profiles',
    filter: `id=eq.${user.id}`
  }, (payload) => {
    updateBalance(payload.new.balance)
  })
  .subscribe()
```

```
// Listen for new notifications
supabase
  .channel('notifications')
  .on('postgres_changes', {
    event: 'INSERT',
    schema: 'public',
    table: 'notifications',
    filter: `user_id=eq.${user.id}`
  }, (payload) => {
    addNotification(payload.new)
    toast.success(payload.new.title)
  })
  .subscribe()

// Admin: listen for new pending transactions
// (admin layout only)
supabase
  .channel('pending-transactions')
  .on('postgres_changes', {
    event: 'INSERT',
    schema: 'public',
    table: 'transactions',
    filter: `status=eq.pending`
  }, (payload) => {
    // increment pending counter, show alert
  })
  .subscribe()
```

STEP 10 — GLOBAL CSS & THEME

src/app/globals.css

```
@import "tailwindcss";

:root {
  --gold: #f59e0b;
  --gold-light: #fbbf24;
  --gold-dark: #d97706;
  --accent: #7c3aed;
  --bg-primary: #080b12;
  --bg-card: rgba(255,255,255,0.04);
  --border: rgba(255,255,255,0.08);
}
```

```
* { -webkit-tap-highlight-color: transparent; }
body { background: var(--bg-primary); overscroll-behavior: none; }
input[type="number"]::-webkit-outer-spin-button,
input[type="number"]::-webkit-inner-spin-button { -webkit-appearance: none; }

/* Marquee */
@keyframes marquee { from { transform: translateX(100%) } to { transform: transla
.animate-marquee { animation: marquee 20s linear infinite; }

/* Safe area */
.pb-safe { padding-bottom: max(env(safe-area-inset-bottom), 1rem); }

/* Custom scrollbar */
::-webkit-scrollbar { width: 3px; }
::-webkit-scrollbar-thumb { background: var(--gold-dark); border-radius: 10px; }

/* Glass card */
.glass-card {
  background: var(--bg-card);
  backdrop-filter: blur(20px);
  border: 1px solid var(--border);
  border-radius: 16px;
}

/* Gold button */
.btn-gold {
  background: linear-gradient(135deg, #f59e0b, #d97706);
  color: #000;
  font-weight: 700;
  border-radius: 12px;
  transition: all 0.15s;
  box-shadow: 0 4px 20px rgba(245,158,11,0.25);
}
.btn-gold:active { transform: scale(0.97); }
```

STEP 11 — PWA CONFIGURATION

public/manifest.json

```
{
  "name": "JMD Online Book",
  "short_name": "JMD Book",
```

```
  "description": "Your trusted online gaming platform",
  "start_url": "/home",
  "display": "standalone",
  "background_color": "#080b12",
  "theme_color": "#f59e0b",
  "orientation": "portrait",
  "icons": [
    { "src": "/icon-192.png", "sizes": "192x192", "type": "image/png" },
    { "src": "/icon-512.png", "sizes": "512x512", "type": "image/png" }
  ]
}
```

In `src/app/layout.tsx` add viewport meta and manifest link. Create simple gold "J" icon images at 192x192 and 512x512 using canvas or inline SVG saved as PNG.

STEP 12 — SECURITY HARDENING (FREE)

Add these without any paid services:

1. **Rate limiting** — Use Supabase DB to track request counts per IP per time window
2. **CSRF protection** — Next.js built-in + SameSite cookies from Supabase SSR
3. **Input sanitization** — Zod schemas on every API route (throw on invalid input)
4. **SQL injection** — Never raw SQL, always Supabase client (parameterized)
5. **File upload validation** — Check MIME type + max 5MB for screenshots
6. **Admin role check** — Verify via DB on every admin API route (don't trust client)
7. **Sensitive data** — Never return password hashes, service keys, or OTP tokens to client
8. **Balance operations** — Always use `update_balance` DB function (atomic, prevents race conditions)
9. **HTTPS** — Vercel enforces this automatically
10. **Headers** — Add security headers in `next.config.ts` :

```
const securityHeaders = [
  { key: 'X-DNS-Prefetch-Control', value: 'on' },
  { key: 'X-Frame-Options', value: 'SAMEORIGIN' },
  { key: 'X-Content-Type-Options', value: 'nosniff' },
  { key: 'Referrer-Policy', value: 'strict-origin-when-cross-origin' },
  { key: 'Permissions-Policy', value: 'camera=(), microphone=()' },
```

```
]
```

STEP 13 — DEPLOYMENT (FREE — VERCEL)

GitHub setup:

```
git init && git add . && git commit -m "initial: JMD Online Book production build
gh repo create jmd-online-book --private && git push -u origin main
```

Vercel:

1. Go to vercel.com → Import GitHub repo
2. Add all `.env.local` vars in Vercel Environment Variables
3. Deploy → auto-deploys on every `git push`
4. Free custom domain OR use `jmd-book.vercel.app`

Supabase Storage buckets:

Run in Supabase dashboard:

```
INSERT INTO storage.buckets (id, name, public) VALUES ('screenshots', 'screenshot
CREATE POLICY "Auth users can upload" ON storage.objects FOR INSERT WITH CHECK (a
CREATE POLICY "Public read" ON storage.objects FOR SELECT USING (bucket_id = 'scr
```

STEP 14 — ADMIN INITIAL SETUP

After deployment, run in Supabase SQL:

```
-- Promote your user account to admin (replace with your user's auth.users id)
UPDATE public.profiles SET role = 'admin' WHERE phone = 'YOUR_PHONE_NUMBER';
```

Then log in at `/login`, navigate to `/admin/dashboard`.

STEP 15 — FINAL CHECKLIST

Before considering this done, verify:

- Login and registration flow works end-to-end
- OTP sent via Resend email and verified
- Deposit form submits → appears in admin pending queue
- Admin can approve → user balance updates in realtime
- Withdraw form submits → appears in admin queue
- Referral code visible in profile, shareable link works
- Registration with referral code creates commission on deposit approval
- All routes protected by middleware (auth required)
- Admin routes blocked for non-admin users
- Mobile nav works on all 5 tabs
- No console errors in production build
- `next build` passes with zero errors
- Lighthouse mobile score > 85

IMPLEMENTATION ORDER

Execute in this order to avoid dependency issues:

1. Bootstrap (Step 0)
2. Supabase DB schema (Step 1c)
3. Env vars + Supabase clients (Steps 1a, 1b, 2)
4. Types (Step 3)
5. Middleware (Step 4)
6. Stores (Step 5)
7. Global CSS + layout shells (Step 10 + layout files)
8. Auth pages — login, register, OTP (Step 7 auth section)
9. API routes (Step 6)

10. Main pages — home, wallet, deposit, withdraw, transactions (Step 7 main section)
11. Reusable components (Step 8)
12. Profile, referral, notifications pages
13. Admin panel (Step 7 admin section)
14. Realtime subscriptions (Step 9)
15. PWA config (Step 11)
16. Security headers (Step 12)
17. Deploy (Step 13)
18. Verify checklist (Step 15)

RULES FOR THE AI CODING AGENT

- **Never write stub functions.** Every function must be fully implemented.
- **Never use `// TODO` comments.** If something needs doing, do it now.
- **Never import from packages not in the npm install list** without adding them first.
- **Always handle errors** — every API route must return proper HTTP status codes.
- **Always validate** — use Zod on every API input.
- **Mobile first** — all UI designed for 375px width, scales up gracefully.
- **Dark theme only** — background `#080b12`, cards `rgba(255,255,255,0.04)`.
- **Gold accent** — primary action color is always `#f59e0b`.
- **Test each feature** before moving to the next one.
- **Run `next build`** before declaring completion. Zero errors required.

START NOW. Begin with Step 0 (bootstrap), then proceed in order.